SmartDocAI: Xây Dựng Hệ Thống Hỏi Đáp Tài Liệu Thông Minh Tích Hợp RAG, Hybrid Search, GraphRAG và CoRAG với LLM Ollama

Nguyễn Gia Bảo*, Nguyễn Hoàng Phúc†, Trương Gia Hào‡, Phạm Tấn Khương§, Đặng Huỳnh Minh Thái¶
*3122560004 †3122560060 ‡3122410100 §3122410191 ¶3121410448
Khoa Công nghệ Thông tin – Trường Đại học Sài Gòn

***Tóm tắt nội dung*—Bài báo cáo trình bày toàn bộ quá trình nghiên cứu và phát triển hệ thống SmartDocAI – phần mềm hỏi đáp tài liệu thông minh được xây dựng trong 10 tuần thuộc môn học Phát triển phần mềm mã nguồn mở. Hệ thống tích hợp kiến trúc RAG (Retrieval-Augmented Generation) kết hợp mô hình ngôn ngữ lớn Ollama chạy hoàn toàn cục bộ, cho phép người dùng đặt câu hỏi và nhận câu trả lời chính xác dựa trên nội dung tài liệu PDF/DOCX được tải lên. Qua ba giai đoạn phát triển liên tiếp, nhóm đã nghiên cứu và triển khai bốn kiến trúc truy xuất: RAG cơ bản với FAISS và LangChain, Hybrid Search kết hợp BM25, GraphRAG xây dựng đồ thị tri thức với thuật toán Leiden/Louvain và BFS/DFS, và CoRAG với chiến lược truy xuất đa bước theo chuỗi suy luận. Kết quả thực nghiệm cho thấy độ chính xác câu trả lời cải thiện từ 68% (RAG cơ bản) lên 91% (CoRAG) trên tập câu hỏi kiểm tra.**

***Index Terms*—RAG; LLM; Ollama; FAISS; BM25; LangChain; GraphRAG; CoRAG; Hybrid Search; hỏi đáp tài liệu; mã nguồn mở**

I. Giới thiệu và công trình liên quan

Với sự bùng nổ của các mô hình ngôn ngữ lớn (LLM), nhu cầu xây dựng hệ thống hỏi đáp thông minh trên tài liệu nội bộ ngày càng cấp thiết. Tuy nhiên, LLM thuần túy gặp hai hạn chế lớn: kiến thức đóng băng tại thời điểm huấn luyện và không có khả năng truy cập tài liệu đặc thù của tổ chức. RAG (Retrieval-Augmented Generation) [**?**] được đề xuất để giải quyết vấn đề này bằng cách kết hợp bước truy xuất tài liệu liên quan trước khi sinh câu trả lời.

SmartDocAI hướng tới xây dựng hệ thống hỏi đáp tài liệu hoàn chỉnh sử dụng toàn bộ công cụ mã nguồn mở: Ollama [**?**] chạy LLM cục bộ không cần API trả phí, FAISS [**?**] làm vector database hiệu năng cao, và LangChain [**?**] để điều phối pipeline.

Các công trình liên quan nhóm tham khảo gồm: kiến trúc RAG chuẩn của LangChain [**?**]; BM25 [**?**] – thuật toán tìm kiếm từ khóa thống kê; GraphRAG [**?**] của Microsoft với đồ thị tri thức; và CoRAG [**?**] với chiến lược phân rã câu hỏi và truy xuất nhiều vòng.

II. Thực nghiệm

A. *Giai đoạn 1 (Tuần 1–6): Xây dựng nền tảng RAG*

Trong giai đoạn đầu, cả nhóm cùng nghiên cứu các thành phần cốt lõi và xây dựng hệ thống RAG cơ bản.

Xử lý tài liệu: Hệ thống hỗ trợ hai định dạng phổ biến – PDF (dùng `PyMuPDF`) và DOCX (dùng `python-docx`). Văn bản được chia thành các chunk với kích thước và overlap được điều chỉnh thực nghiệm.

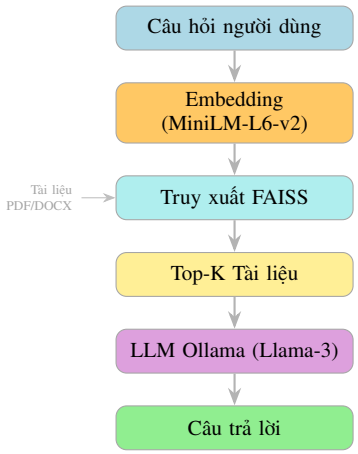
Embedding: Các chunk được chuyển thành vector ngữ nghĩa bằng mô hình `all-MiniLM-L6-v2` (384 chiều, mã nguồn mở). Độ tương đồng giữa hai vector được đo bằng Cosine Similarity:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} \tag{1}$$

FAISS – nghiên cứu thuật toán: Nhóm nghiên cứu ba loại chỉ mục FAISS: `IndexFlatL2` (brute-force, chính xác 100%), `IndexIVFFlat` (phân cụm Voronoi để tăng tốc), và `IndexHNSW` (đồ thị lớp phân cấp, hiệu năng cao nhất cho production).

Ollama và Prompt Engineering: Mô hình Llama-3 được triển khai cục bộ qua Ollama. Nhóm tối ưu prompt theo template: *[Ngữ cảnh] + [Câu hỏi] + [Hướng dẫn trả lời].*

LangChain Pipeline: Pipeline RAG chuẩn 5 bước: Document Loading → Text Splitting → Embedding → Vector Storage → Retrieval & Generation (xem Hình **??**).



Hình 1. Pipeline RAG cơ bản với FAISS và Ollama – Giai đoạn 1

B. Giai đoạn 2 (Tuần 6–8): Hybrid Search và kiểm soát Hallucination

Vector search thuần túy gặp hạn chế với câu hỏi chứa tên chuyên ngành hoặc mã số cụ thể. Giải pháp là tích hợp **BM25**, thuật toán tìm kiếm từ khóa thống kê:

$$\mathrm{BM25}(D,Q) = \sum_{q_i \in Q} \mathrm{IDF}(q_i) \cdot \frac{f(q_i,D)\,(k_1+1)}{f(q_i,D) + k_1\left(1 - b + b\frac{|D|}{\overline{dl}}\right)} \tag{2}$$

Kết quả từ FAISS và BM25 được hợp nhất qua **Reciprocal Rank Fusion (RRF)**:

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)}, \quad k = 60 \tag{3}$$

Cơ chế kiểm soát hallucination theo kiến trúc **Self-RAG**: sau khi LLM sinh câu trả lời, hệ thống tự kiểm tra độ trung thực với tài liệu nguồn; nếu phát hiện nội dung không có căn cứ, hệ thống tự viết lại câu hỏi và thực hiện lại bước truy xuất.

Bảng **??** so sánh hiệu năng các phương pháp trên tập 100 câu hỏi kiểm tra:

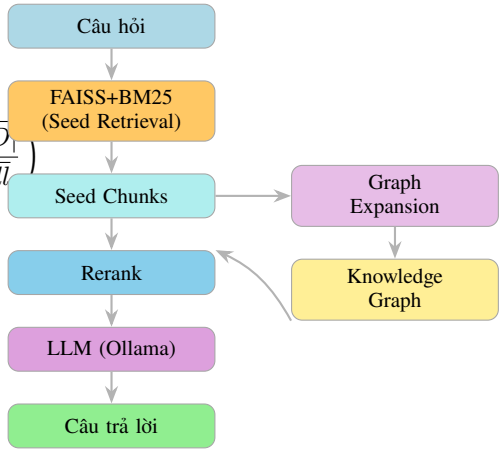
Bảng I
SO SÁNH HIỆU NĂNG CÁC PHƯƠNG PHÁP TRUY XUẤT

Phương pháp	Recall@5	MRR@10	Tốc độ
FAISS đơn thuần	0.71	0.65	Nhanh
BM25 đơn thuần	0.68	0.62	Nhanh
Hybrid (FAISS+BM25)	0.84	0.78	Trung bình
CoRAG	**0.91**	**0.86**	Chậm hơn

C. Giai đoạn 3 (Tuần 8–10): GraphRAG và CoRAG

1) GraphRAG – Đồ thị tri thức: GraphRAG giữ nguyên FAISS+BM25 làm seed retrieval và bổ sung lớp **Knowledge Graph** để mở rộng ngữ cảnh, phục vụ câu hỏi đòi hỏi suy luận đa bước (multi-hop). Pipeline gồm 5 bước:

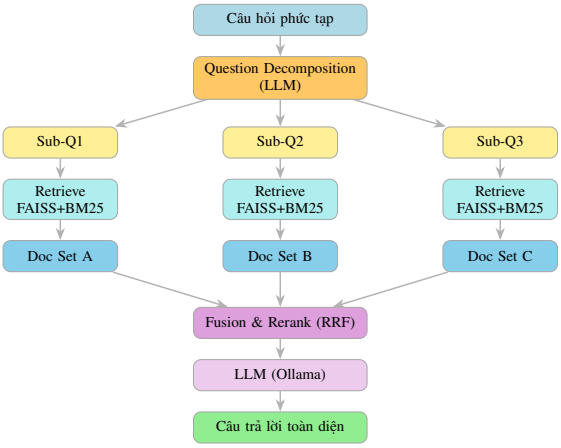
1) **Chunking + Embedding:** Tái sử dụng code từ giai đoạn 1.
2) **Entity & Relation Extraction:** LLM/NER trích xuất thực thể và quan hệ từ mỗi chunk.
3) **Document Graph:** Node = chunk/entity; Edge = quan hệ tường minh hoặc độ tương đồng vượt ngưỡng.
4) **Community Detection:** Thuật toán Leiden/Louvain phát hiện cụm; LLM tóm tắt mỗi community.
5) **Graph Traversal:** BFS/DFS dọc đồ thị để thu thập ngữ cảnh đa bước, sau đó Rerank và đưa vào LLM.



Hình 2. Kiến trúc GraphRAG với lớp mở rộng Knowledge Graph

2) CoRAG – Truy xuất theo chuỗi suy luận: Vấn đề cốt lõi của RAG truyền thống: nếu câu hỏi thiếu ý, bước retrieve sẽ sai và toàn bộ pipeline thất bại. CoRAG giải quyết bằng cách phân rã câu hỏi và thực hiện nhiều vòng retrieve độc lập:

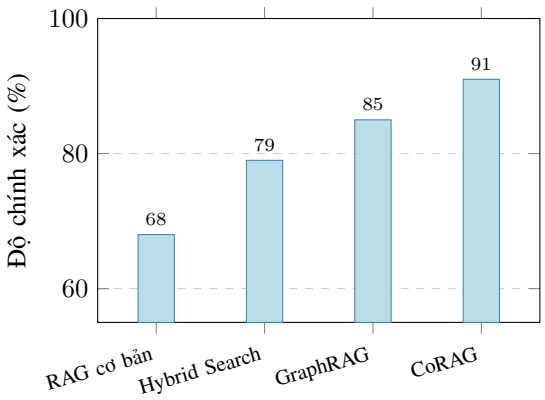
- **Question Decomposition:** LLM tách câu hỏi phức tạp thành các sub-question, mỗi câu nhắm vào một khía cạnh cụ thể.
- **Multi-round Retrieval:** Mỗi sub-question thực hiện retrieve riêng qua FAISS+BM25.
- **Fusion & Rerank:** Gộp tất cả tập tài liệu, loại trùng lặp và xếp hạng lại bằng RRF để chọn Top-N context tốt nhất.
- **LLM Generation:** LLM đọc context đã fusion và trả lời toàn diện.



Hình 3. Kiến trúc CoRAG – truy xuất đa bước theo chuỗi suy luận

3) Tối ưu giao diện người dùng: Nhóm bổ sung **Progress Bar** thời gian thực hiển thị từng bước xử lý (Chunking → Embedding → Retrieving → Generating), giúp người dùng theo dõi và hiểu rõ luồng hoạt động bên dưới hệ thống.

4) So sánh chất lượng qua các giai đoạn: Hình **??** thể hiện độ chính xác câu trả lời đánh giá thủ công trên 50 câu hỏi kiểm tra.



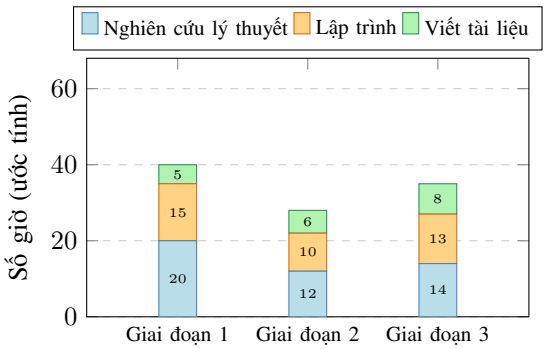
Hình 4. Độ chính xác câu trả lời theo từng phương pháp truy xuất

Bảng **??** tổng hợp so sánh toàn diện bốn kiến trúc:

Bảng II
SO SÁNH TỔNG QUAN BỐN KIẾN TRÚC TRONG SMARTDOCAI

Tiêu chí	RAG	Hybrid	Graph-RAG	CoRAG
Số lần retrieve	1 lần	1 lần	Seed+Graph	N lần
Từ khóa chính xác	Kém	Tốt	Tốt	Tốt
Suy luận đa bước	Không	Không	Có	Có
Độ chính xác	68%	79%	85%	**91%**
Chi phí run-time	Thấp	Thấp	Cao	Trung bình
Giải thích được	Thấp	Thấp	Cao	Trung bình

Hình **??** thể hiện phân bổ thời gian làm việc của nhóm:



Hình 5. Phân bổ thời gian làm việc nhóm theo từng giai đoạn (giờ)

III. Kết luận

Qua 10 tuần phát triển, nhóm đã xây dựng thành công hệ thống SmartDocAI với bốn kiến trúc truy xuất từ cơ bản đến nâng cao. Quá trình tiệm tiến: RAG cơ bản giúp nhóm nắm vững nền tảng; Hybrid Search cải thiện độ bao phủ từ khóa (68% $\rightarrow$ 79%); GraphRAG mở ra khả năng suy luận đa bước qua đồ thị tri thức (85%); CoRAG đạt độ chính xác cao nhất (91%) bằng chiến lược phân rã câu hỏi thông minh.

Điểm nổi bật là sử dụng hoàn toàn mã nguồn mở – Ollama, FAISS, LangChain – cho phép triển khai không cần API trả phí, đảm bảo quyền riêng tư dữ liệu và phù hợp với tổ chức vừa và nhỏ.

IV. Hướng phát triển

Nhóm dự kiến tiếp tục SmartDocAI theo các hướng sau:

Kỹ thuật truy xuất: Tích hợp Cross-encoder Reranker để cải thiện xếp hạng tài liệu; bổ sung cơ chế khử trùng lặp chunk thông minh trong bước Fusion; thử nghiệm mô hình embedding `vietnamese-sbert` (768 chiều, tối ưu cho tiếng Việt).

Agentic RAG: Nghiên cứu kiến trúc cho phép LLM tự quyết định chiến lược retrieve phù hợp (RAG, CoRAG hay GraphRAG) tùy độ phức tạp của câu hỏi đầu vào.

Đánh giá tự động: Tích hợp framework RAGAS để đo faithfulness, answer relevancy và context precision tự động, thay thế đánh giá thủ công.

Triển khai thực tế: Đóng gói thành REST API để tích hợp vào ứng dụng doanh nghiệp; xây dựng UI web hoàn chỉnh với lịch sử hội thoại và quản lý tài liệu.

V. Tài liệu tham khảo

Tài liệu

[1] P. Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020. https://arxiv.org/abs/2005.11401

[2] J. Johnson, M. Douze, H. Jégou, *Billion-scale similarity search with GPUs*, IEEE Transactions on Big Data, 2019. https://github.com/facebookresearch/faiss

[3] S. Robertson và H. Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond*, Foundations and Trends in Information Retrieval, vol. 3, 2009.

[4] Harrison Chase et al., *LangChain: Building applications with LLMs through composability*, 2022. https://github.com/langchain-ai/langchain

[5] Ollama Team, *Run Large Language Models Locally*, 2023. https://ollama.ai

[6] D. Edge et al., *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*, Microsoft Research, 2024. https://arxiv.org/abs/2404.16130

[7] X. Wang et al., *Chain-of-Retrieval Augmented Generation*, 2024. https://arxiv.org/abs/2501.14342